

# Construcción de una UI a *workflows*

# Qué debe resolver una UI conectada a n8n

Una interfaz conectada a n8n no solo debe verse bien. También **debe ayudar a que el *workflow* reciba datos claros, completos y útiles**. Algunos aspectos claves son:

## 1. Guía para el usuario

La interfaz debe pedir solo la información necesaria y mostrar instrucciones claras.

## 2. Feedback inmediato

El usuario debe saber si el formulario se está enviando, si hubo un error o si el proceso terminó correctamente.

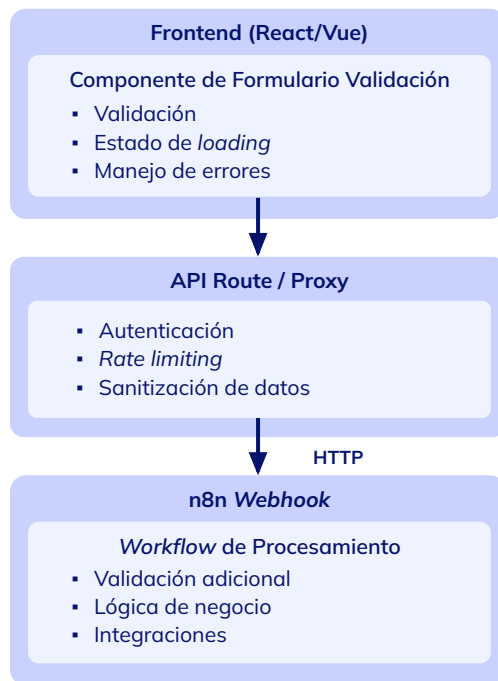
## 3. Validación de datos

Antes de enviar información a n8n, la interfaz puede validar campos obligatorios, formato de *email* o valores incompletos.

## 4. Seguridad básica

Siempre que sea posible, la interfaz no debería exponer directamente URLs sensibles de webhooks o credenciales.

# Anatomía de una interfaz conectada a n8n



## Entrada y salida de datos

Cuando una interfaz se conecta con n8n, puede funcionar de dos maneras: como punto de entrada de datos o como espacio para mostrar una salida del *workflow*.

| Tipo             | Qué significa                                      | Ejemplo                                                  |
|------------------|----------------------------------------------------|----------------------------------------------------------|
| Entrada de datos | La interfaz envía información hacia n8n            | Un formulario envía datos de un <i>lead</i>              |
| Salida de datos  | La interfaz muestra una respuesta generada por n8n | Un mensaje confirma que el registro fue creado           |
| Entrada y salida | La interfaz envía datos y recibe un resultado      | Un chat envía una consulta y muestra una respuesta de IA |

# Ejemplo Completo: Formulario de *Onboarding*

## Requisitos:

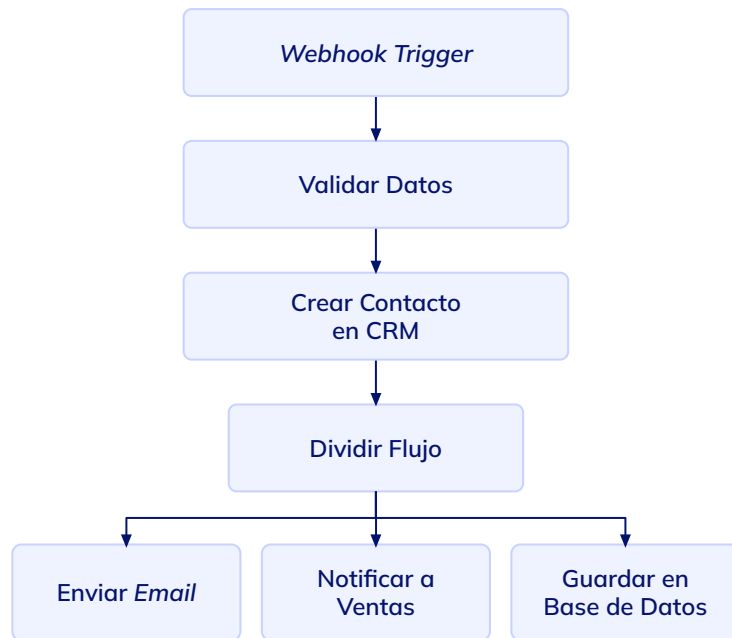
- Formulario *multi*-paso para *onboarding* de clientes.
- Integración con n8n para procesar datos.
- Envío de *email* de bienvenida.
- Creación de cuenta en CRM.
- Notificación a equipo de ventas.

Antes de construir la interfaz, conviene definir qué hará n8n con los datos recibidos.

## Preguntas clave

- ¿Qué datos debe recibir el *workflow*?
- ¿Qué campos son obligatorios?
- ¿Qué validaciones debe realizar?
- ¿Qué herramientas se van a actualizar?
- ¿Qué respuesta debe devolver n8n?
- ¿Qué debe ver el usuario al finalizar?

## Paso 1: Diseñar el *Workflow* en n8n



### Configuración del *Webhook Trigger*:

- *Path*: /onboarding
- *Method*: POST
- *Response*: JSON con *status* y mensaje



## Qué vamos a construir en el código

El ejemplo permite ver cómo una interfaz puede:

- organizar los datos que cargará el usuario;
- dividir el formulario en pasos;
- guardar temporalmente la información ingresada;
- mostrar estados de carga y error;
- enviar los datos hacia una ruta de conexión;
- mostrar una confirmación al finalizar.

## Lectura sugerida del código

| Parte del código | Qué permite hacer                       |
|------------------|-----------------------------------------|
| FormData         | Define qué datos se van a recopilar     |
| useState         | Guarda datos, errores y estado de envío |
| handleSubmit     | Envía el formulario                     |
| renderStep()     | Muestra cada paso del formulario        |
| StepSuccess()    | Confirma que el proceso terminó         |

## Paso 2: Crear el *Frontend* (React)

### Archivo: *components/OnboardingForm.tsx*

```
'use client'
import { useState } from 'react'
import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { Label } from '@components/ui/label'
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from '@components/ui/card'
import { Alert, AlertDescription } from '@components/ui/alert'
import { CheckCircle, Loader2 } from 'lucide-react'

interface FormData {
  firstName: string
  lastName: string
  email: string
  phone: string
  companyName: string
  companySize: string
  industry: string
  goals: string[]
  budget: string
  timeline: string
}
```



...

```
type Step = 1 | 2 | 3 | 4
export default function OnboardingForm() {
  const [currentStep, setCurrentStep] = useState<Step>(1)
  const [formData, setFormData] = useState<FormData>({
    firstName: '', lastName: '', email: '', phone: '',
    companyName: '', companySize: '', industry: '',
    goals: [], budget: '', timeline: ''
  })
  const [isSubmitting, setIsSubmitting] = useState(false)
  const [error, setError] = useState<string | null>(null)
  const updateFormData = (field: keyof FormData, value: any) => {
    setFormData(prev => ({ ...prev, [field]: value }))
  }
  const nextStep = () => { if (currentStep < 4) setCurrentStep((currentStep + 1) as Step) }
  const prevStep = () => { if (currentStep > 1) setCurrentStep((currentStep - 1) as Step) }
  const handleSubmit = async () => {
    setIsSubmitting(true); setError(null)
    try {
      const response = await fetch('/api/onboarding', {
        method: 'POST', headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(formData)
      })
    }
  }
```

...

...

```
    if (!response.ok) throw new Error('Error al procesar el onboarding')
    await response.json()
    setCurrentStep(4)
  } catch (err) {
    setError(err instanceof Error ? err.message : 'Error desconocido')
  } finally { setIsSubmitting(false) }
}

const renderStep = () => {
  switch (currentStep) {
    case 1: return <Step1 formData={formData} updateFormData={updateFormData} />
    case 2: return <Step2 formData={formData} updateFormData={updateFormData} />
    case 3: return <Step3 formData={formData} updateFormData={updateFormData} />
    case 4: return <StepSuccess />
    default: return null
  }
}

return (
  <div className="min-h-screen bg-gradient-to-br from-blue-50 to-indigo-100 flex items-center justify-center p-4">
    <Card className="w-full max-w-2xl">
      <CardHeader>
        <CardTitle>Onboarding de Cliente</CardTitle>
        <CardDescription>Paso {currentStep} de 3</CardDescription>
        <div className="w-full bg-gray-200 rounded-full h-2 mt-4">
```

...

...

```

    <div className="bg-blue-600 h-2 rounded-full transition-all duration-300" style={{ width: `${(currentStep / 3)
* 100}%` }} />

    </div>
  </CardHeader>
  <CardContent>
    {error && (<Alert variant="destructive" className="mb-4"><AlertDescription>{error}</AlertDescription></Alert>)}
    {renderStep()}
    {currentStep < 4 && (
      <div className="flex justify-between mt-6">
        <Button variant="outline" onClick={prevStep} disabled={currentStep === 1 || isSubmitting}>Anterior</Button>
        {currentStep < 3 ? (
          <Button onClick={nextStep}>Siguiete</Button>
        ) : (
          <Button onClick={handleSubmit} disabled={isSubmitting}>
{isSubmitting ? (<<Loader2 className="mr-2 h-4 w-4 animate-spin" />Procesando...</>) : 'Completar Onboarding'}
          </Button>
        )}
      </div>
    )}
  </CardContent>
</Card>
</div>
)
}

```

...

...

```
function Step1({ formData, updateFormData }: any) {
  return (
    <div className="space-y-4">
      <h3 className="text-lg font-semibold">Información Personal</h3>
      <div className="grid grid-cols-2 gap-4">
        <div><Label htmlFor="firstName">Nombre</Label><Input id="firstName" value={formData.firstName} onChange={(e) =>
updateFormData('firstName', e.target.value)} required /></div>
        <div><Label htmlFor="lastName">Apellido</Label><Input id="lastName" value={formData.lastName} onChange={(e) =>
updateFormData('lastName', e.target.value)} required /></div>
      </div>
      <div><Label htmlFor="email">Email</Label><Input id="email" type="email" value={formData.email} onChange={(e) =>
updateFormData('email', e.target.value)} required /></div>
      <div><Label htmlFor="phone">Teléfono</Label><Input id="phone" type="tel" value={formData.phone} onChange={(e) =>
updateFormData('phone', e.target.value)} required /></div>
    </div>
  )
}

function Step2({ formData, updateFormData }: any) {
  return (
    <div className="space-y-4">
      <h3 className="text-lg font-semibold">Información de Empresa</h3>
      <div><Label htmlFor="companyName">Nombre de Empresa</Label><Input id="companyName" value={formData.companyName}
onChange={(e) => updateFormData('companyName', e.target.value)} required /></div>
      <div><Label htmlFor="companySize">Tamaño de Empresa</Label>

```

...

...

```

<select id="companySize" className="w-full p-2 border rounded" value={formData.companySize} onChange={(e) =>
updateFormData('companySize', e.target.value)} required>
  <option value="">Seleccionar...</option>
  <option value="1-10">1-10 empleados</option>
  <option value="11-50">11-50 empleados</option>
  <option value="51-200">51-200 empleados</option>
  <option value="201+">201+ empleados</option>
</select>
</div>
<div><Label htmlFor="industry">Industria</Label><Input id="industry" value={formData.industry} onChange={(e) =>
updateFormData('industry', e.target.value)} required /></div>
</div>
)
}

function Step3({ formData, updateFormData }: any) {
  const goalOptions = ['Automatizar procesos', 'Reducir costos', 'Mejorar eficiencia', 'Escalar operaciones', 'Integrar
sistemas']
  const toggleGoal = (goal: string) => {
    const currentGoals = formData.goals
    if (currentGoals.includes(goal)) updateFormData('goals', currentGoals.filter((g: string) => g !== goal))
    else updateFormData('goals', [...currentGoals, goal])
  }
  return (
    <div className="space-y-4">

```

...

...

```
<h3 className="text-lg font-semibold">Necesidades y Objetivos</h3>
<div>
  <Label>Objetivos (selecciona todos los que apliquen)</Label>
  <div className="space-y-2 mt-2">
    {goalOptions.map(goal => (
      <label key={goal} className="flex items-center space-x-2">
        <input type="checkbox" checked={formData.goals.includes(goal)} onChange={() => toggleGoal(goal)}
className="rounded" />
        <span>{goal}</span>
      </label>
    ))}
  </div>
</div>
<div>
  <Label htmlFor="budget">Presupuesto Mensual</Label>
  <select id="budget" className="w-full p-2 border rounded" value={formData.budget} onChange={(e) =>
updateFormData('budget', e.target.value)} required>
    <option value="">Seleccionar...</option>
    <option value="<$500">Menos de $500</option>
    <option value="$500-$2000">$500 - $2,000</option>
    <option value="$2000-$5000">$2,000 - $5,000</option>
    <option value="$5000+">Más de $5,000</option>
  </select>
</div>
</div>
```

...

...

```
<Label htmlFor="timeline">Timeline de Implementación</Label>
  <select id="timeline" className="w-full p-2 border rounded" value={formData.timeline} onChange={(e) =>
updateFormData('timeline', e.target.value)} required>
    <option value="">Seleccionar...</option>
    <option value="urgent">Inmediato (1-2 semanas)</option>
    <option value="short">Corto plazo (1 mes)</option>
    <option value="medium">Mediano plazo (2-3 meses)</option>
    <option value="long">Largo plazo (3+ meses)</option>
  </select>
</div>
</div>
)
}

function StepSuccess() {
  return (
    <div className="text-center py-8">
      <CheckCircle className="w-16 h-16 text-green-500 mx-auto mb-4" />
      <h3 className="text-2xl font-bold mb-2">Onboarding Completo!</h3>
      <p className="text-gray-600 mb-4">Gracias por completar el proceso de onboarding. Hemos enviado un email de
bienvenida con los próximos pasos.</p>
      <p className="text-sm text-gray-500">Nuestro equipo de ventas se pondrá en contacto contigo en las próximas 24
horas.</p>
    </div>
  )
}
```

## Qué datos se envían a n8n

Cuando el usuario completa el formulario, la interfaz reúne la información cargada y la envía en formato JSON. Este formato permite que n8n identifique cada campo y lo use dentro del *workflow* para validar campos, crear registros, enviar mensajes, guardar información o activar nuevas ramas.

## Ejemplo de datos enviados:

```
{
  "firstName": "Ana",
  "lastName": "García",
  "email": "ana@empresa.com",
  "phone": "+54 351 0000000",
  "companyName": "Empresa Demo",
  "companySize": "11-50",
  "industry": "Servicios",
  "goals": ["Automatizar procesos", "Integrar sistemas"],
  "budget": "$500-$2000",
  "timeline": "short"
}
```



### Paso 3: Crear *API Route* (Proxy a n8n)

```
import { NextResponse } from 'next/server'

const N8N_WEBHOOK_URL = process.env.N8N_ONBOARDING_WEBHOOK_URL

export async function POST(request: Request) {
  try {
    const body = await request.json()
    if (!body.email || !body.firstName || !body.lastName) {
      return NextResponse.json({ error: 'Campos requeridos faltantes' }, { status: 400 })
    }
    const response = await fetch(N8N_WEBHOOK_URL!, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },

      body: JSON.stringify({ ...body, timestamp: new Date().toISOString(), source: 'onboarding-form' })
    })
    if (!response.ok) throw new Error('Error al procesar en n8n')
    const result = await response.json()
    return NextResponse.json(result)
  } catch (error) {
    console.error('Onboarding error:', error)
    return NextResponse.json({ error: 'Error al procesar onboarding' }, { status: 500 })
  }
}
```

## Usar un proxy para proteger la conexión

En lugar de exponer directamente la URL del *webhook* en la interfaz, puede usarse una capa intermedia llamada proxy o API Route.

### ¿Para qué sirve?

- Evita mostrar públicamente la URL real del *webhook*.
- Permite validar datos antes de enviarlos a n8n.
- Permite agregar información adicional, como fecha, origen o identificador del formulario.
- Centraliza el manejo de errores.
- Mejora la seguridad de la integración.

## Qué hace n8n después de recibir los datos

Una vez que n8n recibe los datos desde la interfaz, el *workflow* puede ejecutar distintas acciones.

| Acción en n8n         | Ejemplo                                                         |
|-----------------------|-----------------------------------------------------------------|
| Validar información   | Verificar que el <i>email</i> no esté vacío                     |
| Transformar datos     | Pasar el <i>email</i> a minúsculas                              |
| Guardar registros     | Crear un <i>lead</i> en Supabase o Airtable                     |
| Integrar herramientas | Actualizar un CRM                                               |
| Enviar mensajes       | Mandar <i>email</i> de bienvenida                               |
| Notificar equipos     | Avisar por Slack o Telegram                                     |
| Usar IA               | Clasificar el <i>lead</i> o generar una respuesta personalizada |

## Respuesta del *workflow* a la interfaz

Después de procesar los datos, n8n puede devolver una respuesta a la interfaz. Esa respuesta permite informar al usuario qué ocurrió.

### Ejemplo de respuesta exitosa

```
{  
  "status": "success",  
  "message": "El onboarding fue registrado  
correctamente"  
}
```

### Ejemplo de respuesta con error

```
{  
  "status": "error",  
  "message": "Faltan campos obligatorios"  
}
```

# Buenas prácticas antes de publicar una interfaz

## Entornos: prueba y producción

Cuando una interfaz se conecta con *workflows* reales, conviene diferenciar entre pruebas y uso productivo. Esto permite **evitar que datos de prueba lleguen a herramientas reales o que una automatización incompleta se comparta con usuarios finales**.

Una misma solución puede tener distintos entornos según el nivel de validación necesario.

| Entorno        | Para qué sirve                              | Ejemplo                                                              |
|----------------|---------------------------------------------|----------------------------------------------------------------------|
| Desarrollo     | Crear y probar cambios                      | Formulario conectado a un <i>webhook</i> de prueba                   |
| <i>Staging</i> | Validar el flujo completo antes de publicar | Datos de prueba similares a los reales                               |
| Producción     | Uso real por usuarios o equipos             | <i>Workflow</i> conectado a CRM, <i>email</i> y base de datos reales |

# Versionar cambios en la interfaz

Cuando una interfaz evoluciona, el versionado permite guardar cambios, revisarlos y volver atrás si algo falla.

```
# 1. Crear rama de feature
git checkout -b feature/onboarding-form
# 2. Desarrollar y testear localmente
# ... hacer cambios ...
# 3. Commit frecuentes
git add .
git commit -m "feat: add onboarding form steps 1-2"
# 4. Push a repositorio
git push origin feature/onboarding-form
# 5. Crear Pull Request en GitHub
# 6. Review de código
# 7. Merge a main
# Esto trigger deploy automático en Vercel/Netlify
# 8. Verificar en producción
```

# Identificar versiones estables

Cuando una interfaz ya fue probada, puede marcarse como una versión estable. Esto ayuda a distinguir entre cambios en prueba y versiones listas para usar con *workflows* productivos.

## Usa versionado semántico (semver):

```
# Version format: MAJOR.MINOR.PATCH
# Example: 1.2.3
# Crear tag para release
git tag -a v1.0.0 -m "Release 1.0.0: Initial onboarding system"
git push origin v1.0.0
```

Cuando incrementar:

- MAJOR: Cambios que rompen compatibilidad
- MINOR: Nuevas features compatibles
- PATCH: Bug fixes



# Rollback

Si una interfaz publicada genera errores o envía datos incorrectos al *workflow*, es importante poder volver a una versión anterior.

## Opción 1: *Rollback* en Vercel/Netlify

- Ve al *dashboard* de *deployment*
- Encuentra el *deployment* previo estable
- Click en "*Promote to Production*"



## Opción 2: Git *Revert*

```
# Revertir último commit
git revert HEAD
git push origin main
# Revertir a commit específico
git revert <commit-hash>
git push origin main
```

## Opción 3: Desplegar versión específica

```
# Checkout a tag estable
git checkout v1.0.0
# Force push (usar con cuidado)
git push origin main --force
```

# Publicar una interfaz conectada a n8n

# Preparación para Producción

## Checklist antes de compartir la interfaz

Antes de compartir una interfaz conectada a n8n, revisa que:

- Los campos obligatorios están claros.
- La validación funciona correctamente.
- El *webhook* de n8n recibe los datos.
- Las variables de entorno están configuradas.
- La URL del *webhook* no queda expuesta si se usa proxy.
- Los mensajes de éxito y error son comprensibles.
- Las credenciales no están visibles en el código.
- Se probaron casos con datos incompletos.
- Se revisó qué ocurre si falla una integración.
- El diseño se visualiza correctamente en distintos dispositivos.

## Optimización de *Build*

```
// next.config.js
module.exports = {
  // Optimizaciones de producción
  reactStrictMode: true,
  swcMinify: true,
  // Optimización de imágenes
  images: {
    domains: ['tu-dominio.com'],
    formats: ['image/avif', 'image/webp'],
  },
  // Variables de entorno públicas
  env: {
    NEXT_PUBLIC_API_URL: process.env.NEXT_PUBLIC_API_URL,
  },
}
```

# Deploy

## Deploy en Lovable

Lovable maneja el *deploy* automáticamente:

1. Click en "*Deploy to Production*".
2. Lovable crea:
3. GitHub *Actions workflow*.
4. Dominio en *lovable.app*.
5. *SSL certificate*.
6. Cada *push a main* → *autodeploy*.

## Deploy en Vercel

```
# Instalar Vercel CLI
npm i -g vercel
# Login
vercel login
# Deploy a preview
vercel
# Deploy a production
vercel --prod
Vercel auto-deploys desde GitHub:
Push a cualquier rama → Preview deployment
Push/merge a main → Production deployment
```

## Deploy en Replit

Replit *hosting* es automático:

1. Cada proyecto tiene URL pública.
2. Para *always-on*: Enable "Always On" en plan pago.
3. Para dominio personalizado: *Settings* > *Custom Domain*.

